MZQA

STANDALONE APPLICATION DOCUMENTATION

AI Analyst

System Architecture & Operations

AI Analyst is a self-contained equity-research application that runs a dialectical, multi-agent investment committee over a deterministic valuation engine, a machine-learned quantitative desk, and a read-only fundamentals warehouse. This volume documents *how the application is built and run*: the service topology, the LangGraph committee (its prepare/debate split, the Advocate · Challenger · Auditor tribunal and sector specialists, and the Lead synthesis), the multi-provider LLM layer, the screener and idea tools, the quant desk, the REST surface, and setup.

Companion volumes: Quantitative & ML Models, *qlib Integration*, and *SEC & EDINET Metrics Mapping & Reconciliation*.

Contents

1 Overview

AI Analyst is the investment-committee application of the MZQA equity platform, carved out to run on its own. It carries all the code it needs; the one thing it does *not* carry is the market and fundamentals data — it connects to a pre-built, read-only PostgreSQL warehouse (the `xbrl_sec` data layer, schema `sec`) produced by the ingestion pipeline documented separately.

The application turns a question — *"what is this worth, and what should I expect from it?"* — into a structured, evidence-anchored answer. It exposes several ways to run:

- **Single-stock committee** — the full tribunal plus an institutional report for one ticker.
- **Group / industry verdict** — one relative-value deliberation that ranks the constituents of a GICS sector or an ad-hoc set of names.
- **Idea generation** — a deterministic screener, a natural-language (LLM) screen, and a one-click *value + sentiment* scanner.
- **Quant desk** — a machine-learned return forecast, a forward covariance, and portfolio optimization, exposed as its own tab.

Technology stack.

Frontend	Next.js 14, React 18, TypeScript, Tailwind CSS (port 3027; REST only)
Backend	FastAPI (async), Uvicorn (port 8027); blocking work on worker threads
Orchestration	LangGraph `StateGraph` — the committee state machine
LLM layer	5 providers (DeepSeek, OpenAI, Anthropic, Moonshot, Gemini), chosen per request
Quant layer	Microsoft `qlib` (LightGBM, scikit-learn, cvxpy) — lean in-process embed
Data layer	read-only PostgreSQL `xbrl_sec` warehouse (schema `sec`)
Numerics	pandas, numpy, scipy (deterministic valuation, WACC, comps)

Three backend packages carry the logic: `ai_analyst/committee` (the tribunal graph, valuation, and evidence assembly), `api` (the REST routers and the `api/quant` modeling layer), and `xbrl_sec` (data access, the macro-cycle models, and the LLM client).

2 System architecture

The frontend is a thin client: it collects inputs and renders results. All financial logic and every LLM call live in the backend. The backend is stateless with respect to the application — its only persistent dependency is the shared warehouse; user-owned state (custom analyst types, provider API keys) is kept client-side in the browser.

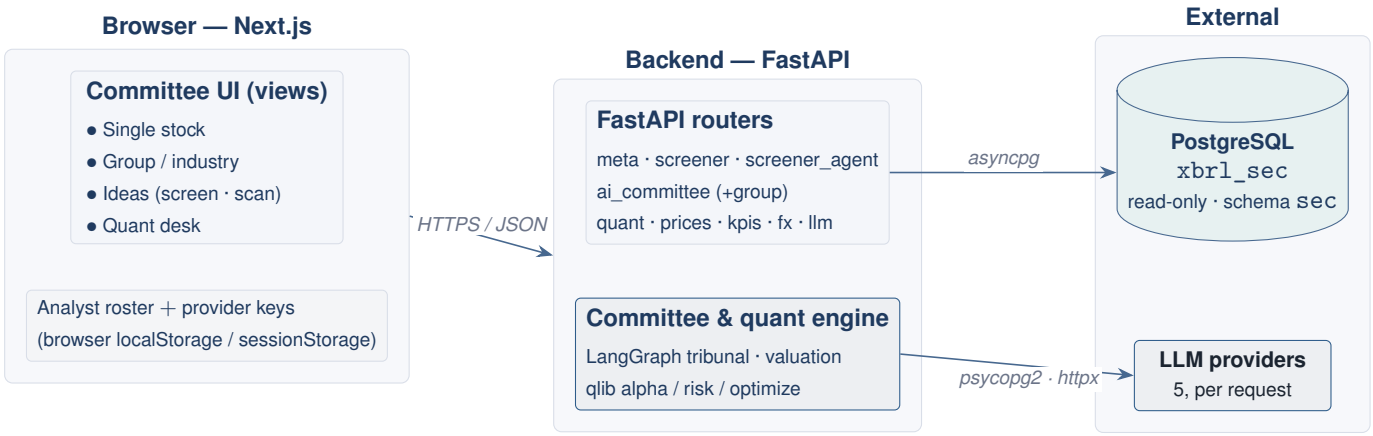

Figure 1. High-level architecture. The browser talks only to the backend; the backend holds all financial logic and is the sole client of Postgres and the LLM providers. The user's provider key travels per request and is never persisted server-side.

2.1 Repository layout

```
AI_Analyst/
|- backend/
|  |- llm_providers.py          # 5-provider registry (stdlib-only, sys.path root)
|  |- api/                      # FastAPI: main.py + routers + db + api/quant modeling
|  |- ai_analyst/committee/     # LangGraph tribunal, valuation, group, evidence
|  \- xbrl_sec/                 # data layer (DB, LLM client, macro cycle, MD&A, news)
|- frontend/                    # Next.js 14 app - the tabbed committee UI
|- docs/                        # this document set
|- start_committee.bat          # one-click launcher (backend :8027 + frontend :3027)
\- .env.example                 # config template (defaults work for a local Postgres)
```

3 The investment committee (LangGraph)

The committee is a `StateGraph` over a single `InvestmentCommitteeState` (a `TypedDict`). It splits into a *prepare* phase and a *debate* phase. A deterministic gate runs *before* any LLM, so unanalyzable data never reaches the agents. If the gate passes, a deterministic engine assembles all the numbers and the evidence nodes attach context; only then does the dialectical tribunal argue over — and tilt assumptions on — that fixed evidence. The Lead synthesizes a probability-weighted fair value and decides whether another debate round is warranted (capped at three).

> **Why the split matters**
> The prepare phase is *provider-independent*: the deterministic packet, the macro/news/institutional evidence, the data-quality report, and the qlib quant signals are all computed once. This lets several LLM providers debate the *same* prepared evidence concurrently, and keeps every figure the tribunal cites reproducible.

3.1 Nodes

Node	Role
`completeness_check`	Ticker resolves? Standardized fundamentals present for the requested years? Sets `is_data_complete` (a hard gate).
`dq_validation`	Checks core accounting identities; sets `is_dq_passed` + `dq_errors`. Advisory by default.
`financial_analysis_engine`	Deterministic packet: WACC, comps, SOTP, cash-flow/ROIC, reverse-DCF. No LLM.
`news_macro / institutional`	Parallel evidence: macro regime + news sentiment; 13F ownership.
`data_quality_agent`	Read-only DQ report over the five data layers, folded into the evidence bundle.
`qlib_signals`	Alpha expected return + percentile, forward vol, factor exposures (prepare phase, shared).
`advocate / challenger / auditor`	The built-in tribunal (LLM): argue their stance over the fixed evidence.
`specialist` nodes	Built at runtime (e.g. Quality-of-Earnings Auditor, Quantitative Factor Analyst) + user mandates.
`lead_analyst`	Reconciles the debate, extracts scenarios, decides `decision_ready`.
`memo_generator`	Bilingual institutional memo (EN / DE).

3.2 The data-governance gate (advisory by default)

Completeness is always a hard stop — with no fundamentals the engine cannot run. The accounting-identity DQ check, however, is *advisory by default*: the committee proceeds and surfaces the findings as a warning. Strict blocking is opt-in via `config.dq_enforce` (the UI's *strict data-governance* toggle).

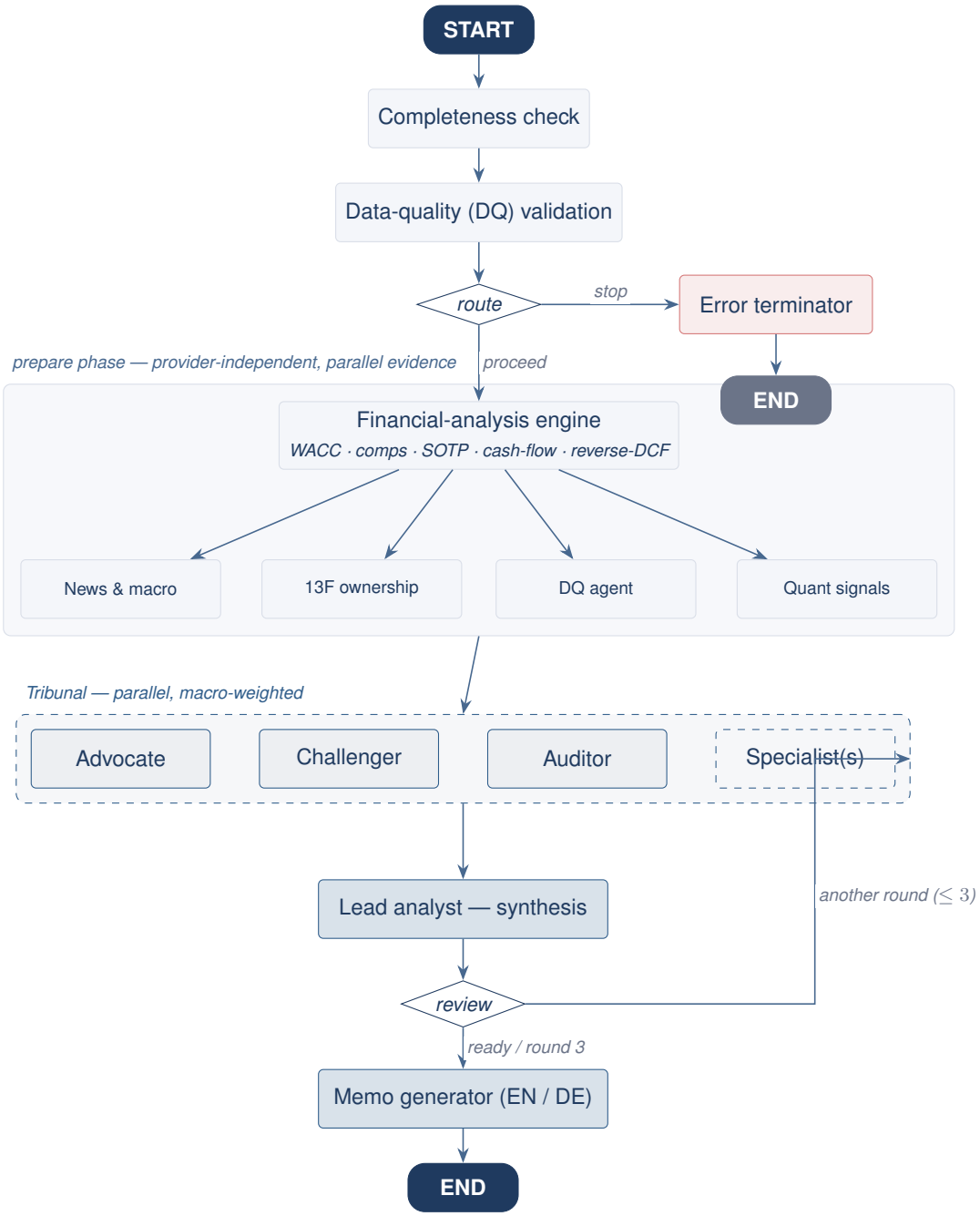

Figure 2. Committee topology. The prepare phase gates on data, builds the deterministic packet, and fans out four evidence nodes; the debate phase runs the Advocate · Challenger · Auditor tribunal plus specialists, the Lead synthesis, and the bilingual memo.

```python
def route_data_validation(state):
    if not state.get("is_data_complete"):
        return "terminate_error"                    # unanalyzable -> hard stop
    if get_config(state).get("dq_enforce") and not state.get("is_dq_passed"):
        return "terminate_error"                    # strict mode only
    return "analyze_data"                           # advisory: proceed, warn later
```

The HTTP layer mirrors this: it returns **422** only when the data is incomplete or strict mode blocked the run; otherwise it returns **200** with a `dq_warning` object the UI renders as an amber banner.

4 The multi-provider LLM layer

Every agent node routes through a provider-agnostic LLM layer built on a single registry (`llm_providers.py`, stdlib-only). Five providers are supported and selected *per request*; the user's API key travels with the request and is never persisted server-side.

Provider	Wire dialect	Notes
DeepSeek	openai	default provider
OpenAI (ChatGPT)	openai	
Moonshot (Kimi)	openai	reasoning model; thinking can be disabled to avoid empty completions
Gemini (Google)	openai	via Google's OpenAI-compatibility endpoint
Anthropic (Claude)	anthropic	dedicated adapter on the official SDK (no OpenAI-compatible endpoint)

Two wire dialects cover all five: the `openai` dialect serves `POST {base_url}/chat/completions` with a uniform message/tool shape (a small capability record captures where sampling params, JSON mode, and the max-tokens field differ), and the `anthropic` dialect is a dedicated adapter. The layer is used two ways: a *narrative* tier (plain-text theses and the memo) and a *structured* tier (`with_structured_output` against Pydantic schemas for scenario / weight / verdict extraction). A host allowlist is the security boundary that stops a user-supplied `base_url` from walking off with the key. Offline / no-key runs fall back to deterministic placeholders, so every deterministic path remains runnable and testable without any provider.

5 Screener, ideas, and the group verdict

The screener filters the US/JP universe on fundamentals (deterministic SQL) and offers a natural-language screen (LLM → filters). Three idea sub-features use the LLM layer differently:

- **Value + sentiment scan** — one LLM call per name for MD&A tone, on the user's selected provider, blended with the qlib alpha and regime-conditioned factor-IC weights into a 0–100 interest score.
- **Prompt screen** — the prompt→filters translation fans out across every keyed provider; each surfaces a candidate set (the per-name "found by" badges), and the union is ranked once.
- **Group ranking** — a deterministic quantitative composite (z-scored P/E, EV/EBITDA, P/B, FCF yield, growth, margins). No LLM is needed for the ranking; the LLM only writes the optional narrative, on the selected provider.

For an industry or a screen, the group endpoint resolves the universe to a capped ticker set, builds the deterministic composite as a seed, and (when a key is present) refines it with a single structured verdict that attaches per-name stance and rationale.

6 The quant desk

The quant desk is a lean, in-process integration of `qlib`: a cross-sectional alpha model (return prediction), a factor-structured forward covariance, dual portfolio optimization (a native optimizer *and* qlib's, selectable at runtime), and a walk-forward out-of-sample backtest with Fama–French benchmarking. It is exposed through a `/api/quant/*` router and a *Quant* UI tab, and fed into the committee as the `qlib_signals` evidence node and a Quantitative Factor Analyst specialist. The mathematics of each model is specified in the *Quantitative & ML Models* volume; its engineering integration in the *qlib Integration* volume.

7 REST API

Base `http://127.0.0.1:8027`, all under `/api`; handlers offload blocking work to worker threads.

Endpoint	Method	Purpose
`/api/meta/filters`	GET	GICS sector / industry-group lists
`/api/screener/run`	POST	Deterministic structured screen
`/api/screener/ai`	POST	Natural language → filters
`/api/screener/agent/value-sentiment`	POST	Value + sentiment scanner
`/api/ai/committee`	POST	Single-stock committee (also `/prepare`, `/debate`, `/iterate`)
`/api/ai/committee/group`	POST	Group relative-value verdict
`/api/quant/{backends, alpha, risk, optimize, backtest}`	--	Quant desk (mathematics in the ML Models volume)
`/api/{prices, kpis, company, fx, llm}`	GET	Market data, KPIs, company, FX, provider catalogue
`/api/healthz`	GET	Liveness + DB check

8 Setup and running

The application works on defaults if a local Postgres `xbrl_sec` database is running and the connection and (optionally) a provider key are configured.

```
cd AI_Analyst

# 1) Backend venv + lean deps
py -3 -m venv .venv
.\.venv\Scripts\pip install -r backend\requirements.txt

# 2) Frontend deps
cd frontend; npm install; cd ..

# 3) Point at the existing Postgres + keys (once; keys can also be pasted in the UI)
setx PGPASSWORD "your-postgres-password"
setx DEEPSEEK_API_KEY "sk-your-key-here"

# 4) Run both services and open the browser
.\start_committee.bat
```

A token-free, deterministic smoke test of the whole committee engine (no browser, no provider) runs the offline path:

```
$env:PYTHONPATH="$PWD\backend"
.\.venv\Scripts\python -m ai_analyst.committee.run --ticker MSFT --offline --no-news
```

Ports
The standalone launcher defaults to backend `:8027` and frontend `:3027`, separate from the parent platform's defaults. The backend must be started from `backend/` so `qlib` resolves; the quant desk pre-warms the alpha cross-section and the walk-forward backtest at startup, so the first user request is served from cache.

9 Data dependencies and notes

- **Shared, read-only warehouse.** The committee reads standardized fundamentals, market metrics, prices, GICS classification, 13F ownership, macro series, factor loadings, MD&A, and news from the Postgres `xbrl_sec` warehouse (schema `sec`). The application never writes to it. How that warehouse is built — the raw-to-standardized mapping, the metrics, and the reconciliation — is the subject of the *SEC & EDINET Metrics Mapping & Reconciliation* volume.
- **App-owned state.** Deployed analyst types and provider keys live in the browser, precisely because the warehouse is read-only.
- **Latency.** A full single-stock committee run is minutes-scale even in offline / deterministic mode; group and scan runs are seconds-scale.

AI Analyst — a deterministic valuation core, a machine-learned quant desk, a multi-provider reasoning layer, and LangGraph orchestration.